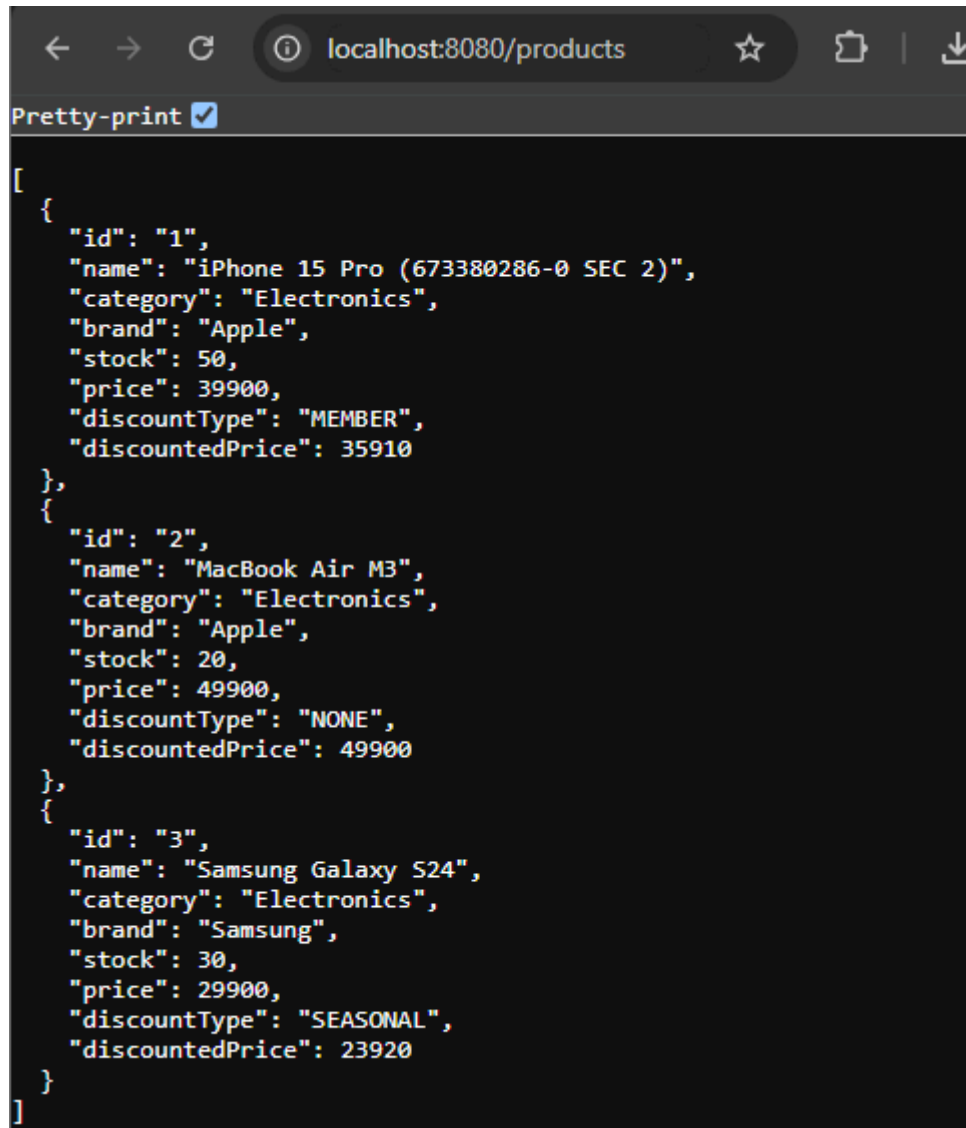
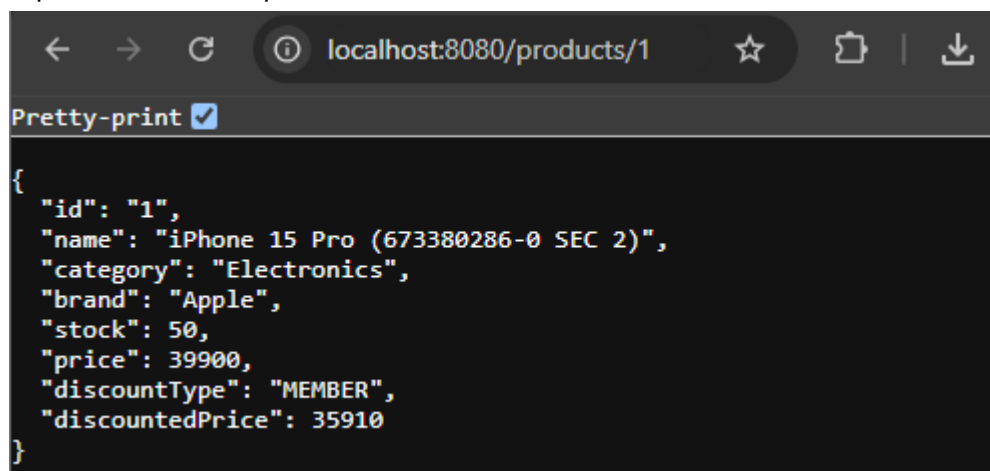


http://localhost:8080/products



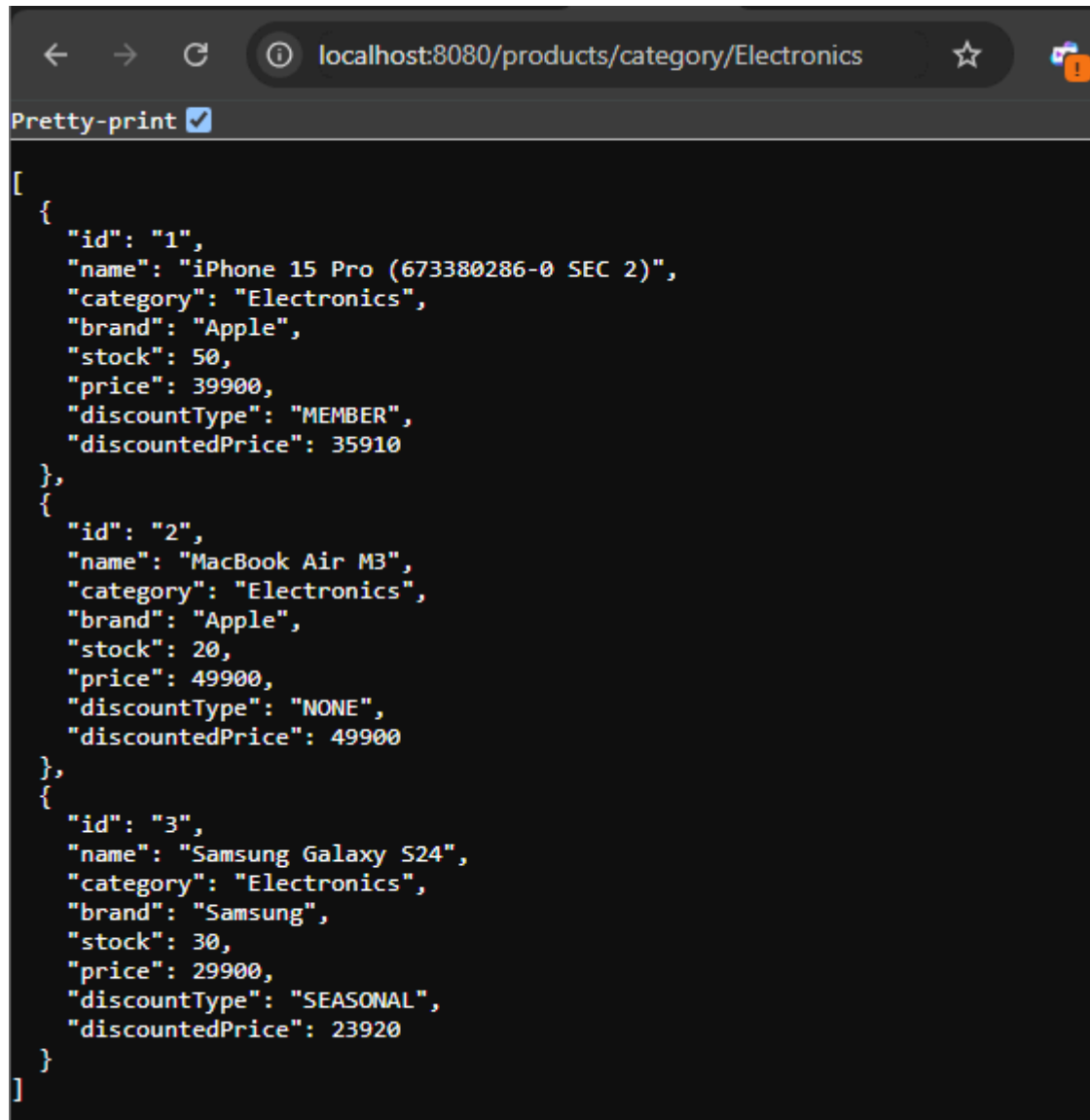
```
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380286-0 SEC 2)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

http://localhost:8080/products/1



```
{
  "id": "1",
  "name": "iPhone 15 Pro (673380286-0 SEC 2)",
  "category": "Electronics",
  "brand": "Apple",
  "stock": 50,
  "price": 39900,
  "discountType": "MEMBER",
  "discountedPrice": 35910
}
```

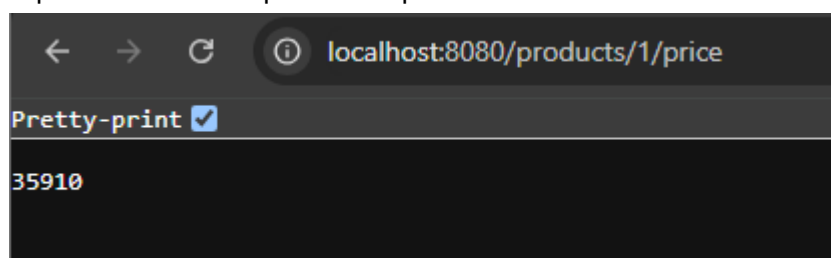
http://localhost:8080/products/category/Electronics



A screenshot of a web browser window. The address bar shows the URL 'localhost:8080/products/category/Electronics'. Below the address bar, there is a 'Pretty-print' button with a checkmark. The main content area displays a JSON array of three product objects. The first object is for an iPhone 15 Pro, the second for a MacBook Air M3, and the third for a Samsung Galaxy S24. Each object contains fields for id, name, category, brand, stock, price, discountType, and discountedPrice.

```
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380286-0 SEC 2)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

http://localhost:8080/products/1/price



A screenshot of a web browser window. The address bar shows the URL 'localhost:8080/products/1/price'. Below the address bar, there is a 'Pretty-print' button with a checkmark. The main content area displays the value '35910'.

```
35910
```

post

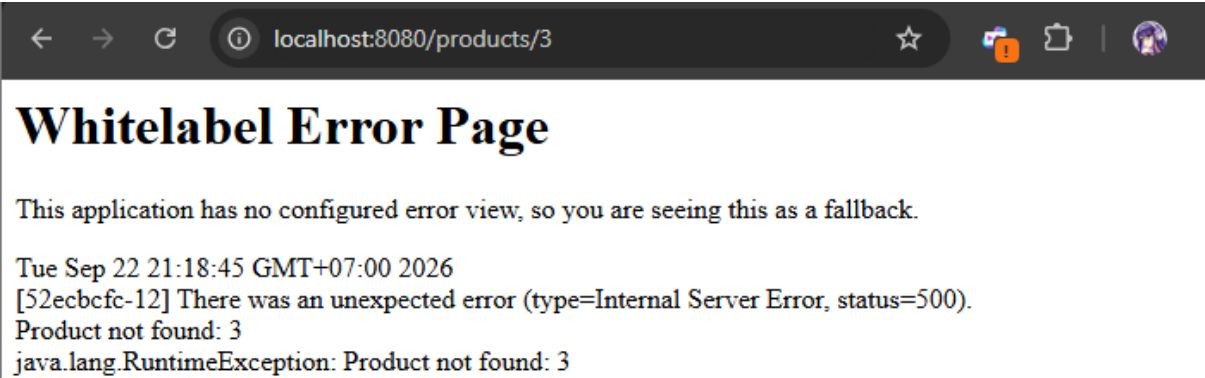
```
below and press Enter to allow pasting.

allow pasting
> fetch('/products', {method: 'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify({name: 'Test 673380286-0', category: 'Electronics', brand: 'Test', stock: 10, price: 999.0, discountType: 'NONE'})}).then(r => r.json()).then(console.log)
< ▶ Promise {<pending>}
▶ {id: 'f3522ecd-3f4c-4e8f-8d97-73aa3733ad3e', name: 'Test 673380286-0', category: 'Electronics', brand: 'Test', stock: 10, ...}
> |
```

DELETE

```
> fetch('/products/3', { method: 'DELETE' }).then(r => console.log('status:', r.status))
< ▶ Promise {<pending>}
status: 200 VM83:1
> |
```

ลองลบ



อธิบาย

1. Reactive Programming vs Blocking — ต่างกันอย่างไร

	Blocking	Reactive
Thread ตอนรอผล	หยุดรอ (block) จนกว่าจะได้ค่า	ไม่หยุดรอ — ไปทำงานอื่นระหว่างรอ
Scalability	1 thread ต่อ 1 request → thread หมดเร็วเมื่อ concurrent สูง	ไม่กีด thread รองรับ request จำนวนมาก (non-blocking I/O)
Return type	ค่าจริงตรงๆ เช่น <code>Product</code>	<code>Mono<Product></code> / <code>Flux<Product></code> — เป็นแค่ "สัญญา" ว่าจะมีค่ามาทีหลัง

เหมาะกับ	งาน CRUD ทั่วไป, traffic ไม่สูง	งาน high-concurrency, streaming data
----------	---------------------------------	--------------------------------------

2. Mono vs Flux — ใช้กรณีไหน และทำไม

Mono<T> — ใช้เมื่อผลลัพธ์มีได้ 0 หรือ 1 ค่า เช่น `findById`, `save`, `delete` (คืน 1 object หรือไม่มีเลย)

Flux<T> — ใช้เมื่อผลลัพธ์มีได้ 0 ถึง N ค่า เช่น `findAll`, `findByCategory` (คืนหลาย object เป็น stream)

3. WebClient — อธิบาย method chain `.get()`, `.uri()`, `.retrieve()`, `.bodyToMono/Flux()`

.get() — เลือก HTTP verb (มี `.post()`, `.delete()`, `.put()` ด้วย) — ยังไม่ส่ง request จริง แค่เริ่มสร้าง request spec

.uri(...) — ระบุ path ปลายทาง รองรับ path variable แบบ `{id}` แทนที่ด้วย argument ถัดไป

.retrieve() — บอกว่า "พร้อมรับ response แล้ว" เป็นจุดที่เริ่ม exchange การเรียกจริงแบบ non-blocking (ทั้งหมดยังไม่เกิดจนกว่าจะ `subscribe()`)

.bodyToMono(T.class) / .bodyToFlux(T.class) — deserialize JSON response body เป็น **Mono<T>** (1 object) หรือ **Flux<T>** (หลาย object) ตาม cardinality ของ endpoint ที่เรียก

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

Repository — เก็บ/ค้นข้อมูลดิบ ห่อด้วย Mono/Flux

```
public class ProductRepository {
    private final Map<String, Product> store = new ConcurrentHashMap<>();

    // คืน Mono ว่าง ถ้าไม่พบ id — ไม่ throw exception ที่นี่ (หน้าที่ของ Service)
    public Mono<Product> findById(String id) {
        Product product = store.get(id);
        return product == null ? Mono.empty() : Mono.just(product);
    }

    // ห่อ collection ทั้งหมดเป็น Flux — คืนได้ 0..N รายการ
    public Flux<Product> findAll() {
        return Flux.fromIterable(store.values());
    }

    // บันทึกแบบ synchronous ภายใน (in-memory) แต่ยัง wrap เป็น Mono
    // เพื่อให้ signature สอดคล้องกับ reactive chain ทั้งระบบ
    public Mono<Product> save(Product product) {
        store.put(product.getId(), product);
        return Mono.just(product);
    }
}
```

```

    public Mono<Void> deleteById(String id) {
        store.remove(id);
        return Mono.empty(); // Mono<Void> = ไม่มี payload ค็น แค่ signal เสร็จ
    }

    // ใช้ operator filter บน Flux ที่ได้จาก findAll() — ไม่ query ซ้ำ
    public Flux<Product> findByCategory(String category) {
        return findAll().filter(p -> p.getCategory().equalsIgnoreCase(category));
    }
}

```

Service — business logic ต่อยอดจาก Repository

@Service

```

public class ProductService {
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) { this.repository = repository; }

    // switchIfEmpty: ถ้า Repository คืน Mono ว่าง ให้แทนที่ด้วย Mono.error
    // ทำให้ผู้เรียก (Controller) ได้ error แทนการได้ null เจ็บๆ
    public Mono<Product> getById(String id) {
        return repository.findById(id)
            .switchIfEmpty(Mono.error(new RuntimeException("Product not found: " +
id)));
    }

    public Flux<Product> getAll() {
        return repository.findAll();
    }

    // business rule: ถ้าไม่ส่ง id มา ให้ generate UUID ก่อนบันทึก
    public Mono<Product> save(Product product) {
        if (product.getId() == null) {
            product.setId(java.util.UUID.randomUUID().toString());
        }
        return repository.save(product);
    }

    public Mono<Void> delete(String id) {
        return repository.deleteById(id);
    }

    public Flux<Product> getByCategory(String category) {
        return repository.findByCategory(category);
    }
}

```

```

// map: แปลงค่าภายใน Mono แบบ synchronous (Product → Double)
// ใช้ getById() เพื่อให้ error-handling (not found) ทำงานร่วมด้วย
public Mono<Double> getDiscountedPrice(String id) {
    return getById(id).map(Product::getDiscountedPrice);
}
}

```

Controller — expose เป็น REST endpoint

```

@RestController
@RequestMapping("/products")
public class ProductController {
    private final ProductService service;

    public ProductController(ProductService service) { this.service = service; }

    @GetMapping public Flux<Product> getAll() { return service.getAll(); }

    @GetMapping("/{id}")
    public Mono<Product> getById(@PathVariable String id) { return service.getById(id); }

    // @RequestBody ถูก deserialize เป็น Product แบบ non-blocking โดย WebFlux เอง
    @PostMapping
    public Mono<Product> save(@RequestBody Product product) { return
service.save(product); }

    @DeleteMapping("/{id}")
    public Mono<Void> delete(@PathVariable String id) { return service.delete(id); }

    @GetMapping("/category/{category}")
    public Flux<Product> getByCategory(@PathVariable String category) { return
service.getByCategory(category); }

    @GetMapping("/{id}/price")
    public Mono<Double> getDiscountedPrice(@PathVariable String id) { return
service.getDiscountedPrice(id); }
}

```